Skill Doctor: A High-Performance, Multi-Layer Security Framework for AI Agent Skill Files

Sayan Chowdhury
Kalaris Labs Research Team
sayan@kalarislabs.com · github.com/KalarisLabs/Skill-Doctor

August 2026 · Preprint · Release v0.2.3

Abstract—The proliferation of autonomous AI agent frameworks—such as Claude Code, OpenAI Codex CLI, Gemini CLI, Cursor, Windsurf, and their successors—has introduced a novel and largely undefended attack surface: the agentic *skill file*. Skill files are structured instruction documents (`SKILL.md`, `.clauderules`, `AGENTS.md`, and Model Context Protocol (MCP) server configurations) that agents dynamically load from public registries, private repositories, and third-party archives at execution time. Once loaded, a skill file executes within the full trust boundary of the agent, inheriting unrestricted access to tools, file systems, APIs, and downstream agents. In this paper, we formalize the attack taxonomy against skill files, synthesizing the OWASP Agentic Skills Top 10 and real-world vulnerability disclosures. We present **Skill Doctor**, a multi-engine security platform built entirely in Rust. By combining memory-safe YARA-X static analysis, native `tree-sitter` cross-file Abstract Syntax Tree (AST) scanning, and Large Language Model (LLM)-powered semantic reasoning, Skill Doctor accurately detects malicious, deceptive, and non-compliant skill files prior to runtime ingestion. Benchmarks demonstrate that Skill Doctor's native single-binary architecture achieves sub-30ms static scan times, significantly outperforming legacy Python-based tools like Cisco Skill Scanner and NVIDIA SkillSpector. Furthermore, we outline the deployment of Skill Doctor as a GitHub Action to enable frictionless registry-level gating, securing the supply chain of the agentic web.

Keywords: AI Security, Agentic Web, Static Analysis, Supply Chain Security, Model Context Protocol (MCP), Rust, Abstract Syntax Trees (AST)

1 Introduction

In modern computational paradigms, AI agents have evolved from isolated conversational interfaces into orchestrated pipelines capable of autonomous execution. These agents rely heavily on dynamic external configurations—referred to collectively as *skill files*—to define their behavior, available tools, and systemic context at runtime. The trust model has fundamentally shifted: an agent does not merely execute explicit user prompts; it autonomously executes instructions defined by any skill file it ingests.

In traditional software ecosystems, malicious binaries require execution permissions, OS-level exploitation, or bypassing sandboxing mechanisms. In the agentic paradigm, a skill file requires only that an agent runtime loads it. This loading process is often automatic, silent, and occurs at scale across millions of automated agent sessions.

The severity of this attack vector has been empirically demonstrated:

- During the ClawHavoc Campaign (2026), 1,184 confirmed malicious skills were discovered in ClawHub, exploiting automatic context injection.
- A survey of 7,000+ public Model Context Protocol (MCP) servers by BlueRock Security revealed that 36.7% were vulnerable to Server-Side Request Forgery (SSRF) attacks due to unvalidated tool schemas [5].
- Research by NVIDIA (2026) indicates that skills bundled with companion scripts (e.g., Python utilities) are 2.12× more likely to contain exploitable command injection vulnerabilities compared to standalone Markdown configurations [3].

Despite these critical risks, defensive tooling remains immature. Existing static scanners rely on highly latent interpreted runtimes, lack cross-file contextual analysis, and operate in isolation without community threat intelligence.

Skill Doctor mitigates these architectural deficiencies by introducing a high-performance, Rust-native platform designed for direct integration into CI/CD pipelines, GitHub Actions, and agent registries.

2 Threat Model and Taxonomy (SDTM-v1)

We define the Skill Doctor Threat Model (SDTM-v1), a synthesis of the OWASP Agentic Skills Top 10, the MCP Top 10, and observed adversarial behavior in public skill registries.

2.1 Classification of Attacks

- **SD-01 · Prompt Injection**: Direct system prompt overrides, ASCII smuggling (zero-width Unicode payloads), and base64-encoded payloads.
- **SD-02 · Command Injection**: Unsanitized parameters passed to critical execution sinks such as `subprocess.run()`, `eval()`, or `pickle.loads()` within companion scripts.
- **SD-03 · Data Exfiltration**: Malicious directives targeting sensitive paths (e.g., `~/.aws/credentials`) and leaking them via HTTP callbacks or DNS exfiltration.
- **SD-04 · Privilege Escalation**: Tool scope violations, such as a localized code-formatting skill silently registering system-wide read permissions.
- **SD-05 · Supply Chain Tampering**: Cryptographic hash mismatches, typosquatting, and `.pyc` cache poisoning within skill bundles.
- **SD-06 · SSRF via Tool Parameters**: Unvalidated URL schemas in tool descriptions designed to coerce the agent into scanning internal network architectures.
- **SD-07 · Tool Poisoning**: Shadow tool registration, wherein a malicious skill registers an obfuscated proxy tool masquerading as a legitimate system utility.
- **SD-08 · Persistent Backdoors**: The injection of persistence mechanisms into auto-loaded environment files (e.g., `.cursorrules`) to survive the deletion of the original skill.
- **SD-09 · Context Window Flooding**: The generation of excessively large stochastic outputs designed to purposefully overflow the agent's context window.
- **SD-10 · Obfuscation and Evasion**: Homoglyph substitution, bidirectional text spoofing, and multi-layer encoding chains.

3 System Architecture and Methodology

To achieve comprehensive security without compromising registry throughput, Skill Doctor utilizes a multi-layer analysis pipeline driven by a highly concurrent Rust backend (`tokio`). The pipeline degrades gracefully, ensuring that static rules trigger even if external LLM providers experience latency.

3.1 The Bundle vs. File Paradigm

A critical limitation of legacy scanners is their isolated, file-by-file analysis approach. A syntactically benign `SKILL.md` can effortlessly mask a maliciously crafted `utils.py` located in the same directory.

Skill Doctor inherently treats skills as *bundles*. Upon ingestion, the static engine utilizes `tree-sitter` bindings to construct a unified Abstract Syntax Tree (AST). This enables cross-file taint analysis, seamlessly tracking variable flow from user inputs in a Markdown configuration to dangerous execution sinks in a companion Python script.

3.2 High-Performance Engine

By relying on `yara-x` (a memory-safe Rust implementation of YARA) and `tree-sitter`, the core scanning engine bypasses the overhead associated with spinning up Python interpreters, achieving near-instantaneous static evaluation.

4 Competitive Analysis and Benchmarks

We benchmarked Skill Doctor `v0.2.0` (Rust) against Cisco Skill Scanner (v1.4) and NVIDIA SkillSpector (v2.1). The evaluation corpus consisted of 1,000 benign skills and 50 synthetically generated malicious skills engineered to evade traditional static analysis.

Testing Environment: Ubuntu 24.04 LTS, 8-Core AMD EPYC, 16GB RAM.

4.1 Execution Latency and Scalability

Metric	Skill Doctor (Rust)	NVIDIA SkillSpector	Cisco Skill Scanner
Startup Overhead	**˜2 ms**	˜450 ms (Python init)	˜1,200 ms (heavy imports)
Scan (Single File)	**˜12 ms**	˜85 ms	˜210 ms
Scan (Bundle - 50)	**˜28 ms**	˜310 ms	˜890 ms

Table 1: Performance benchmarking showing a 30× improvement over Python baselines.

4.2 Detection Efficacy

Metric	Skill Doctor (Rust)	NVIDIA SkillSpector	Cisco Skill Scanner
True Positive Rate	**98.2%**	84.0%	91.5%
False Positive Rate	**1.1%**	3.8%	2.4%
Cross-file Catch	**100%**	0% (Isolated scans)	40% (Partial AST)

Table 2: Detection efficacy rates against SDTM-v1 threats.

5 Registry Gating and GitHub Actions Integration

A standalone Command Line Interface (CLI) is insufficient to secure an entire ecosystem; security must be enforced structurally at the distribution and integration layer.

To actualize this, Skill Doctor is designed to operate natively within CI/CD pipelines. We have officially deployed the system as a GitHub Action (`KalarisLabs/skill-doctor-action@v1`). This integration acts as a "Registry Gate API," enabling public skill registries to implement frictionless, pre-commit webhooks.

Listing 1: Example YAML for GitHub Actions Registry Gating

```yaml
name: Skill Doctor Validation
on: [pull_request]
jobs:
  scan-skill:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validate Skill Payload
        uses: KalarisLabs/skill-doctor-action@v1
        with:
          target: './submitted-skills/'
          fail-on: 'HIGH'
```

By forcing all newly submitted skills through the Skill Doctor GitHub Action, registries can mathematically guarantee that payloads containing known OWASP vulnerabilities are blocked before they are published to end-users.

6 Conclusion

Skill Doctor represents a paradigm shift in autonomous agent security infrastructure. By abandoning legacy interpreted languages in favor of a memory-safe, hyper-performant Rust architecture, it enables real-time security gating for the agentic web. Through its bundle-first analysis philosophy and seamless integration paths via GitHub Actions, Skill Doctor provides the foundational security layer necessary to safely scale multi-agent ecosystems.

References

[1] OWASP Foundation. *Agentic Skills Top 10.* 2026.

[2] OWASP Foundation. *MCP Top 10.* 2026.

[3] NVIDIA Research. *SkillSpector: Static and Semantic Analysis of AI Agent Skill Files.* arXiv, 2026.

[4] Cisco AI Defense. *Skill Scanner: Open-Source Skill Security for AI Agents.* GitHub, 2026.

[5] BlueRock Security. *SSRF in the Wild: A Survey of 7,000 MCP Servers.* 2026.

[6] Alice.io. *Caterpillar: Scanning the Top 50 Skills on Skills.sh.* 2026.